

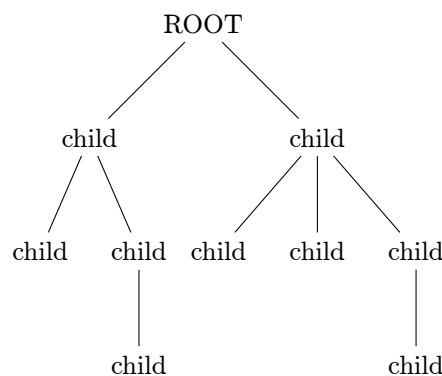
ALBERI GENERALI

INDICE SEZIONE

- ☐ INTRODUZIONE
- ☐ DEFINIZIONI
- ☐ INTERFACCE ITERATOR E ITERABLE
- ☐ INTERFACCIA TREE
- ☐ ALGORITMI
- ☐ VISITE DI ALBERI

INTRODUZIONE

In modo *informale*, un albero è una **collezione di nodi** caratterizzata da una *struttura gerarchica* che si dipana da un nodo **radice** tramite relazioni *padre-figlio*.



OSSERVAZIONI

- Le relazioni padre-figlio costituiscono un insieme di collegamenti minimali che inducono un legame fra tutti i nodi.
- Una lista è un caso estremo di albero con una struttura gerarchica lineare.

CAMPI APPLICATIVI:

- Strutture dati**: mappe, priority queue.
- Esplorazione risorse**: filesysteme, siti di e-commerce.
- Reti di comunicazione**: sistemi distribuiti, sincronizzazione, broadcasting.
- Analisi di algoritmi**: albero della ricorsione.
- Compressione dati**: codici di Huffman.

- [Biologia computazionale](#): alberi filogenetici.

DEFINIZIONI

Diamo la definizione formale di albero (radicato):

ALBERO RADICATO (ROOTED TREE)

Un *albero radicato* T è una collezione di nodi che, se non è vuota, soddisfa le seguenti proprietà:

- Esiste un *nodo speciale* $r \in T$ detto **radice**.
- $\forall v \in T, v \neq r : \exists! u \in T : u$ è **padre** di v e (v è figlio di u).
- $\forall v \in T, v \neq r$ si ha che, **risalendo** di padre in padre si arriva a r (ogni nodo è **discendente dalla radice**).

ELEMENTI DEGLI ALBERI

Vediamo di seguito le definizioni di elementi degli alberi.

ANTENATI

Diciamo che x è *antenato* di y se $x = y$ oppure se x è antenato del padre di y .

DISCENDENTI

Diciamo che x è *discendente* di y se y è antenato di x .

NODI INTERNI

Sono i nodi con ≥ 1 figli.

NODI ESTERNI (FOGLIE)

Sono i nodi senza figli.

SOTTOALBERO CON RADICE v

E' T_v : l'albero formato da tutti i discendenti di v (quindi include v).

ALBERO ORDINATO

Diciamo che T è un *albero ordinato* se per ogni nodo interno $v \in T$ è definito un *ordinamento lineare* tra i figli $u_1, \dots, u_k$ di v .

DEFINIZIONE RICORSIVA

Possiamo dare anche una definizione *ricorsiva* di albero:

ALBERO RADICATO (DEF. RICORSIVA)

Un albero radicato T è una collezione di nodi che, *se non è vuota*, risulta partizionata nel seguente modo:

$$T = \{r\} \cup T_1 \cup \dots \cup T_k$$

Per un qualche $k \geq 0$, dove:

- r è radice con figli $u_1, u_2, \dots, u_k$.
- $\forall i, 1 \leq i \leq k : T_i$ è un albero non vuoto con radice u_i .

ALTRE DEFINIZIONI SUGLI ALBERI

PROFONDITA' DI UN NODO

Possiamo dare due definizioni:

1. $\text{depth}_T(v) = \text{antenati}(v) - 1$.
2. Se $v = r$ allora $\text{depth}_T(v) = 0$, altrimenti $\text{depth}_T(v) = 1 + \text{depth}_T(\text{padre}(v))$.

LIVELLO i

Insieme di nodi a profondità i ($\forall i \geq 0$).

ALTEZZA DI UN NODO

Se v è foglia, allora $\text{height}_T(v) = 0$.

Altrimenti $\text{height}_T(v) = 1 + \max(\text{height}_T(w))$ con w figlio di v .

L'altezza "complessiva" dell'albero sarà quindi data da:

ALTEZZA DI UN ALBERO

$\text{height}(T) = \text{height}_T(r)$ con r radice di T .

Vediamo allora il seguente fatto:

PROPOSIZIONE

Dato un albero T non vuoto, $\text{height}(T) = \max(\text{depth}_T(v))$ con v foglia.

E dimostriamolo:

✓ DIM.

Definiamo $h := \text{height}(T)$ e $d := \max(\text{depth}_T(v))$ con $v \in T$ foglia.

Dimostriamo che $h \geq d$.

- Sia v una foglia a profondità d e sia r la radice di T .
- Allora esiste un percorso da v a r formato dai nodi $u_d = v$ ad altezza 0, u_{d-1} ad altezza ≥ 1 , ..., u_1 ad altezza $\geq d-1$ ed $u_0 = r$ ad altezza $\geq d$.
- Abbiamo quindi, in generale, $\text{height}_T(u_i) \geq d-i \ \forall d \geq i \geq 0$.
- Perciò si ha $h = \text{height}_T(r) = \text{height}_T(u_0) \geq d$.

Dimostriamo ora che $h \leq d$.

- Consideriamo ancora una foglia v a profondità d . Assumiamo per assurdo che $h > d$.
- Essendo h altezza della radice, deve esistere un percorso da una foglia v ad r formato dai nodi $u_0 = v$ ad altezza 0, u_1 ad altezza 1, ..., u_{h-1} ad altezza $h-1$ e $u_h = r$ ad altezza h .
- Ma allora v avrebbe h antenati diversi da se stessa, per cui sarebbe $\text{depth}_T(v) = h > d$, che contraddice il fatto che d è la massima profondità di una foglia.
- Deve quindi necessariamente essere $h \leq d$

Ma era anche $h \geq d$, per cui concludiamo che $h = d$. $\square$

INTERFACCE ITERATOR E ITERABLE

Prima di definire l'interfaccia di un albero definiamo le seguenti due, su cui essa sarà poi basata:

- **Iterator**: un " *cursore* " che permette di enumerare (*scansionare*) gli elementi di una collezione.
- **Iterable**: una collezione "*iterabile*", cioè che rende disponibile un iteratore ai suoi elementi.

```
public interface Iterator<E>{  
  
    // Returns true if the scan of the collection is not over  
    boolean hasNext();  
  
    // Returns the next element in the collection  
    E next();  
}
```

```
public interface Iterable<E>{  
  
    // Returns an iterator of the collection  
    Iterator<E> iterator();  
}
```

L'interfaccia di un albero estenderà quindi *Iterable* in modo da rendere disponibile un *iteratore* con cui scansionare i nodi dell'albero.

INTERFACCIA TREE

L'implementazione tipica di un albero è realizzata come segue:

- Un **puntatore** alla **radice** (unico *punto di accesso*).
- Ogni **nodo** è implementato come un *oggetto a sè stante* (per esempio una classe che implementi l'interfaccia Position) che offre dei metodi per *accedere* al *padre* e ai *figli*.

L'*interfaccia* di un albero (tree) è quindi la seguente:

```

public interface Tree<E> extends Iterable<E>{

    // Returns the number of positions in the tree
    int size();

    // Returns true if the tree contains no positions
    boolean isEmpty();

    // Returns the Position of the root (or null if empty)
    Position<E> root();

    // Returns the Position of p's parent (or null if p is the root)
    Position<E> parent(Position<E> p);

    // Returns an iterable containing p's children
    Iterable<Position<E>> children(Position<E> p);

    // Returns the number of children of p
    int numChildren(Position<E> p);

    // Returns true if p is internal
    boolean isInternal(Position<E> p);

    // Returns true if p is external
    boolean isExternal(Position<E> p);

    // Returns true if p is root
    boolean isRoot(Position<E> p);

    // Returns an iterator to all elements in the tree
    Iterator<E> iterator();

    // Returns an iterable containing all positions in the tree
    Iterable<Position<E>> positions();

}

```

ALGORITMI

PROFONDITA' DI UN NODO

Vediamo un algoritmo ricorsivo per calcolare la profondità di un nodo:

- **INPUT:** $v \in T$
- **OUTPUT:** profondità di v in T .

```
Algorithm depth( $v$ )  
1: if  $T.isRoot(v)$  then  
2:   return 0  
3: else  
4:   return 1 + DEPTH( $T.parent(v)$ )  
5: end if
```

Analizziamone la **complessità**:

- Se d_v è la profondità di v nell'albero T , saranno eseguite $d_v + 1$ *invocazioni ricorsive* di depth, *una per ogni antenato*.
Per cui l'albero della ricorsione ha $d_v + 1$ livelli da un nodo ciascuno.
- Il costo di ogni nodo è $\Theta(1)$.

Concludiamo allora che la complessità di $depth(v)$ è $\Theta(d_v + 1)$.

OSSERVAZIONE

Il $+1$ è giustificato dal fatto che se $d_v = 0$ (v è radice) sono comunque richieste $\Theta(1)$ operazioni (non esiste $\Theta(0)$).

Tuttavia possiamo semplificare la notazione e usare $\Theta(d_v)$ per evidenziare il fatto che comunque la complessità è *proporzionale* alla *profondità* stessa.

Avendo usato come *taglia dell'istanza* proprio la **profondità dell'albero**, le possibili istanze di taglia d sono tutti i *nodi a profondità d* di *qualsiasi albero* T (che abbia nodi a profondità almeno d).

A questo punto ci si potrebbe domandare se sia possibile esprimere la complessità dell'algoritmo usando come *taglia dell'istanza* il **numero di nodi** n .

In questo caso le possibili istanze di taglia n sono tutti i *nodi* appartenenti ad *alberi* T con proprio n nodi ($|T| = n$).

Allora abbiamo:

- **Upper bound:** $O(n)$ perchè in un albero con n nodi, ogni nodo ha profondità $\leq n$.
- **Lower bound:** $\Omega(n)$ perchè in un albero che è una *catena* di n nodi, l'ultimo nodo della catena è una foglia v di profondità $n - 1$.

Concludiamo quindi che in tal caso la complessità al caso pessimo sarebbe $\Theta(n)$.

ALTEZZA DI UN NODO

Vediamo un altro algoritmo ricorsivo, stavolta per il calcolo dell'altezza di un nodo.

- **INPUT:** $v \in T$.
- **OUTPUT:** altezza di v in T .

Algorithm height(v)

```
1:  $h \leftarrow 0$ 
2: for all  $w \in T.children(v)$  do
3:    $h \leftarrow \max\{h, 1 + HEIGHT(w)\}$ 
4: end for
5: return  $h$ 
```

NOTA: il caso base è dato da v foglia, che non ha figli.

Nel for che scansiona i figli di v assumiamo che ciascuno venga generato in tempo $\Theta(1)$.

Analizziamo ora la complessità. L'**albero della ricorsione** associato a $height(v)$:

- Ha **un nodo per ogni** $u \in T_v$ (sottoalbero di radice v).
- Il costo associato al nodo $u \in T_v$ è $\Theta(c_u + 1)$ dove c_u è il **numero di figli** di u .

Allora la complessità di $height(v)$ è data da:

$$\Theta \left(\sum_{u \in T_v} (c_u + 1) \right)$$

Riscriviamo la sommatoria come segue:

$$\sum_{u \in T_v} c_u + \sum_{u \in T_v} 1 = \sum_{u \in T_v} c_u + n_v$$

Dove n_v è il numero di nodi in T_v .

In generale vale il seguente fatto:

$$\sum_{u \in T_v} c_u = n_v - 1$$

E concludiamo allora

$$\sum_{u \in T_v} (c_u + 1) = 2n_v - 1$$

Per cui possiamo dire che la complessità di $height(v)$ è $\Theta(n_v)$.

VISITE DI ALBERI

Diamo la definizione di visita di un albero:

📖 VISITA DI UN ALBERO

Scansione sistematica di *tutti i nodi* dell'albero T che permette di eseguire una qualche operazione (*visita*) ad ogni nodo.

Le visite rappresentano un *design pattern* algoritmico che può essere utilizzato per il calcolo di valori e/o per impostare opportune variabili associate ai nodi.

Per alberi generali studiamo le visite:

- **Preorder**: visita *prima il padre* e *poi* (ricorsivamente) i sottoalberi radicati nei *figli*. In tal modo, le operazioni svolte per un nodo possono *dipendere da quelle svolte per i suoi antenati*.
- **Postorder**: visita *prima* (ricorsivamente) i sottoalberi radicati nei *figli*, *poi il padre*. In questo modo le operazioni svolte per un nodo possono *dipendere da quelle svolte per i suoi discendenti*.

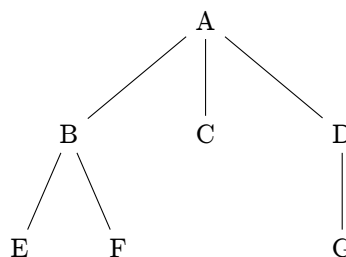
VISITA PREORDER

- **INPUT**: nodo $v \in T$.
- **OUTPUT**: risultante della visita di T_v .

```
Algorithm preorder(v)
1: visita v
2: for all  $w \in T.children(v)$  do
3:   PREORDER(w)
4: end for
```

NOTA: il caso base è dato da v foglia, che non ha figli.

Consideriamo per esempio il seguente albero:



Assumendo che `children()` esamini i figli da sinistra a destra, l'ordine della visita in preorder è **A B E F C D G**.

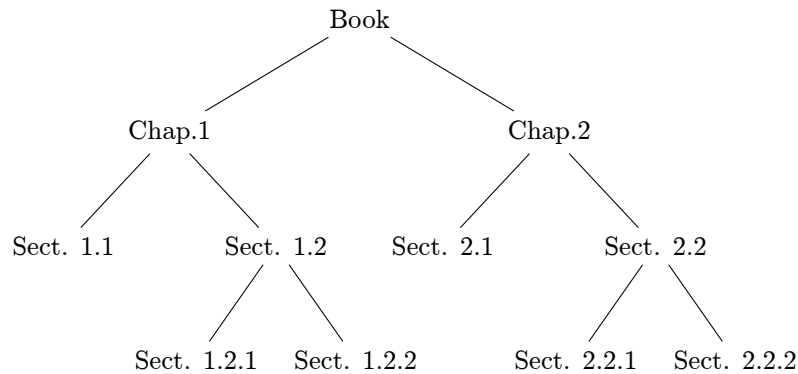
📖 DEFINIZIONE

Sia T albero ordinato e siano $u, v \in T$ due nodi allo stesso livello.

Diciamo che u è **a sinistra** di v (e quindi v è **a destra** di u) se u viene prima di v nella visita in preorder.

ESEMPIO APPLICATIVO

Consideriamo l'indice di un libro rappresentato dal seguente albero:



≡ ESEMPI

Per stampare l'indice di tale libro dobbiamo usare la visita in **preorder**.

Un esempio analogo sarebbe dato dalla stampa della struttura di un **file system** come sequenza di cartelle e file.

ALGORITMO BASATO SULLA VISITA PREORDER

Vogliamo progettare un algoritmo `allDepths` che dato un albero T calcoli la **profondità** di ogni nodo $v \in T$ e la memorizzi in un campo `v.depth`.

Per farlo possiamo adattare la visita in preorder, definendo la visita di un nodo v come segue:

- Se v è **radice**, si imposta la profondità a 0.
- Se v **non è radice**, si imposta la profondità a 1+ la profondità del padre, che è già stata impostata dato che il padre è già stato visitato.

Vediamo l'algoritmo:

- **INPUT:** $v \in T$ e `u.depth` impostato correttamente per u padre di v .
- **OUTPUT:** `z.depth` impostato correttamente $\forall z \in T_v$.

Algorithm allDepths(T,v)

```
1: if T.isRoot(v) then //Visita v
2:   v.depth ← 0
3: else
4:   v.depth ← 1 + T.parent(v).depth
5: end if
6: for all w ∈ T.children(v) do //Visita ricorsivamente i figli
7:   ALLDEPTHS(T,w)
8: end for
```

🔥 OSSERVAZIONI

1. Per impostare il campo depth per *tutti i nodi* di T invochiamo allDepths(T , $T.root()$).
2. L'algoritmo *non restituisce* alcun valore, ma modifica i nodi dell'albero che si considerano quindi *oggetti globali* che sopravvivono all'esecuzione dell'algoritmo.

VISITA POSTORDER

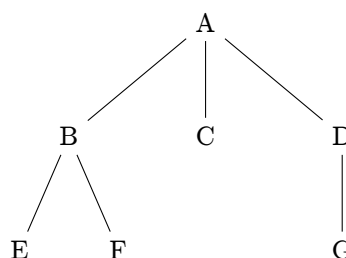
- **INPUT**: nodo $v \in T$.
- **OUTPUT**: risultante della visita di T_v .

Algorithm postorder(v)

```
1: for all w ∈ T.children(v) do
2:   POSTORDER(w)
3: end for
4: visita v
```

NOTA: il caso base è dato da v foglia, che non ha figli.

Consideriamo per esempio il seguente albero:



Assumendo che children() esamini i figli da sinistra a destra, l'ordine della visita in preorder è **E F B C G D A**.

ALGORITMI BASATI SULLA VISITA POSTORDER

≡ ESEMPIO 1

L'algoritmo `height` è un esempio di visita *postorder* dato che l'altezza di un nodo viene calcolata completamente solo dopo aver calcolato quelle dei figli.

≡ ESEMPIO 2

Si consideri un *file system* gerarchico la cui struttura è rappresentata da un albero T dove i nodi *interni* corrispondono alle *cartelle* e i nodi *foglia* ai *file*.

Ogni nodo v ha un campo `v.loc - size` che memorizza lo spazio occupato dal nodo, escludendo quello dei discendenti.

Vogliamo progettare un algoritmo `diskSpace` che dato un tale T calcoli, per ogni nodo $v \in T$, lo spazio aggregato occupato dai suoi discendenti e lo memorizzi in un campo `v.aggr - size`.

Vediamo come implementare l'algoritmo dell'esempio 2.

- **INPUT:** Albero T e nodo $v \in T$, `u.loc - size` impostato $\forall u \in T$.
- **OUTPUT:** `v.aggr - size` e `z.aggr - size` impostato correttamente $\forall z \in T_v$.

Algorithm `diskSpace(T,v)`

```
1: v.aggr-size ← v.loc-size
2: for all  $w \in T.children(v)$  do
3:   v.aggr-size ← v.aggr-size + DISKSPACE(T,w)
4: end for
5: return v.aggr-size
```

COMPLESSITA' VISITE

Sia T un albero costituito da n nodi.

Consideriamo l'*albero della ricorsione* di `preorder(T.root())`:

- Ha n nodi associati alle *invocazioni ricorsive* che sono esattamente *una per ogni nodo di T* .
- Il costo del nodo associato a `preorder(v)` con $v \in T$ generico è $\Theta(t_v + c_v + 1)$.
Dove t_v è il *costo della visita di v* e c_v è il numero di figli di v (stiamo assumendo che i figli siano restituiti in tempo $\Theta(1)$).

Possiamo allora concludere che la complessità di `preorder` è:

$$\Theta \left(\sum_{v \in T} (t_v + c_v + 1) \right) = \Theta \left(n + \sum_{v \in T} t_v \right)$$

NOTA

1. Si ha la *stessa complessità* anche per **postorder** e **inorder** (per gli alberi binari, vedremo).
2. Dall'analisi si conclude che le visite consentono di *enumerare tutti i nodi* di T in tempo *lineare* (l'operazione sul nodo non è considerata, ci stiamo riferendo alla sola enumerazione dei nodi).
3. Se $t_v \in \Theta(1)$ allora abbiamo che *anche la complessità totale* è $\Theta(n)$.